Football Match Prediction — Project Roadmap & Daily Checklist

Goal: Build a working multi-output regression system (goals, shots on target, later corners) for EPL, La Liga, Bundesliga, Serie A.
Pace: ~30–45 min/day, 5 days/week. Adjust as needed — consistency matters more than speed.
How to use this: Create a recurring calendar event "Football ML — Daily Practice" Mon–Fri, same time each day. Copy each day's task into the event description or a to-do app (Microsoft To Do, Notion, Todoist — any works).

WEEK 1 — Environment Setup + First Data Look

Milestone (mini project): Load one season of EPL data and answer 3 basic questions about it in Jupyter.

- **Day 1:** Install Python, create venv, install packages (pandas, numpy, scikit-learn, jupyter, tensorflow). Confirm `(venv)` shows in terminal.
- **Day 2:** Create folder structure (data/raw, data/processed, notebooks, src, models). Download EPL 2023/24 CSV from Football-Data.co.uk into data/raw.
- **Day 3:** Open Jupyter. Load CSV with `pd.read_csv()`. Run `.head()`, `.info()`, `.describe()`. Note down what each column means.
- **Day 4:** Practice basic pandas: filter matches by team, sort by date, find highest-scoring match. (This builds comfort before

feature engineering.)

- **Day 5 (Mini Project):** In a notebook, answer: (1) Which team scored the most home goals? (2) What's the average goals per match? (3) How many matches ended in a draw?

WEEK 2 — Elo Ratings

Milestone: A working Elo rating function that updates after every match, chronologically.

- **Day 1:** Review Elo formula (expected outcome, actual outcome, update rule). Write it out in plain pseudocode on paper first.
- **Day 2:** Write the `expected_score()` function in Python. Test it manually with 2-3 example rating pairs.
- **Day 3:** Write the `update_elo()` function. Test with a win, a draw, a loss scenario — check numbers match your manual calculations from earlier.
- **Day 4:** Loop through your dataset **in date order**, updating Elo for every team after every match. Store each team's Elo history.
- **Day 5 (Mini Project):** Plot one team's Elo rating over the season using matplotlib. Does it rise/fall in ways that make sense (losing streak = drop)?

WEEK 3 — Rolling Form + Rest Days

Milestone: A processed dataset with Elo, last-5 form (home/away split), and rest days — ready for modeling.

- **Day 1:** Learn `.rolling(5).mean()` and why you must `.shift(1)` first (to avoid leaking today's match into today's "form" feature).

- **Day 2:** Build last-5-games rolling average for goals scored/conceded, per team.
- **Day 3:** Split rolling form into home-specific and away-specific versions (a team's home form ≠ away form).
- **Day 4:** Calculate rest days: days since each team's previous match. Handle season-start edge cases (no previous match).
- **Day 5 (Mini Project):** Merge everything into one clean "model-ready" dataframe. Save it to data/processed/. Sanity check: no missing values in your feature columns before the season's 6th matchday.

WEEK 4 — Temporal Splitting + First Baseline Model

Milestone: A working (if simple) baseline model that outputs actual predictions.

- **Day 1:** Learn temporal train/validate/test splitting. Split your data by season (e.g., train on earlier seasons, test on most recent).
- **Day 2:** Learn what "baseline" means and why you need one. Build the simplest possible baseline: predict the training average for every match.
- **Day 3:** Install and learn basics of scikit-learn's `RandomForestRegressor` with `MultiOutputRegressor`.
- **Day 4:** Train your first Random Forest on Elo + form + rest days → predict goals + shots on target.
- **Day 5 (Mini Project):** Calculate MAE for your model vs the baseline. Did it beat the baseline? By how much? Write 2-3 sentences on what you observe.

WEEK 5 — XGBoost + Model Comparison

Milestone: Two tree-based models compared fairly.

- **Day 1:** Install XGBoost. Learn how it differs from Random Forest in plain terms (builds trees one at a time, each correcting the last one's mistakes).
- **Day 2:** Train an XGBoost multi-output regressor on the same features.
- **Day 3:** Compare MAE of Random Forest vs XGBoost, per output (goals, shots on target).
- **Day 4:** Try adjusting 1-2 hyperparameters (e.g., number of trees, max depth) — observe effect on validation MAE.
- **Day 5 (Mini Project):** Write a short "model comparison" summary — which model wins, on which stat, and by how much.

WEEK 6-7 — Neural Network (TensorFlow/Keras)

Milestone: A working multi-output neural network, trained and evaluated.

- **Week 6, Day 1:** Learn Keras basics: Sequential vs Functional API (you'll need Functional API for multi-output).
- **Week 6, Day 2:** Build the input layer + 2 hidden layers (64 → 32 neurons) using the Functional API.
- **Week 6, Day 3:** Add the 4 output heads (Home Goals, Away Goals, Home SoT, Away SoT).
- **Week 6, Day 4:** Compile the model (Adam optimizer, MSE loss). Fit on your training data with a small number of epochs first (test it runs without errors).

- **Week 6, Day 5 (Mini Project):** Get the model training end-to-end, even if results aren't great yet. Save it with `model.save()`.
- **Week 7, Day 1:** Add early stopping (stop training when validation loss stops improving).
- **Week 7, Day 2:** Scale your input features with StandardScaler before training (important for neural nets).
- **Week 7, Day 3:** Retrain with scaling + early stopping. Compare MAE to Week 6 version.
- **Week 7, Day 4:** Compare neural network MAE vs Random Forest vs XGBoost, side by side.
- **Week 7, Day 5 (Mini Project):** Write your "final Phase 1 model comparison" — which approach wins overall? Keep this model as your best Phase 1 model.

WEEK 8 — Live Prediction Pipeline

Milestone: Predict an actual upcoming weekend's matches.

- **Day 1:** Sign up for API-Football (or football-data.org) free tier. Explore how to pull upcoming fixtures.
- **Day 2:** Pull this weekend's fixtures for your chosen league.
- **Day 3:** Write a function that computes current Elo/form/rest-days for each team as of today (using your saved historical data).
- **Day 4:** Feed those features into your saved best model → get predictions.
- **Day 5 (Mini Project):** Predict this weekend's matches. After the matches happen, compare your predictions to actual results. How close were you?

WEEK 9+ — Phase 2: Corners + Scaling to All 4 Leagues

Milestone: Corners added as a 5th output; pipeline expanded to La Liga, Bundesliga, Serie A.

- Add corners column to your targets, retrain.
- Re-run your whole pipeline (Elo, form, rest days, model) on each additional league — reuse your Week 1-8 code, don't rebuild from scratch.
- Optional stretch: explore FBref/soccerdata for xG data to boost accuracy further.

Calendar Setup Suggestion

Create ONE recurring event:

- **Title:** Football ML — Daily Practice
- **Time:** Pick a consistent 30-45 min slot (e.g., 7:00–7:45 AM or after work)
- **Days:** Mon–Fri
- **Description:** Paste in "Current week" section from this doc, update weekly

Third-party option: if you want built-in progress tracking, **Notion** or **Todoist** work well — create one task per day from this checklist, check off as you go. Not required — a calendar reminder alone is enough to stay consistent.

Notes

- If a day's task takes longer than planned, it's fine to spread it over 2 days. The schedule is a guide, not a hard deadline.
- Every "Mini Project" day is a checkpoint — don't skip it, even if rushed. It's how you confirm you actually understood the week, not just copied code.
- Bring code/errors back to Claude as you go — debugging together is part of the process, not a sign you're behind.